

LAPORAN STUDI KASUS

Perbandingan Forward Chaining dan Backward Chaining

Studi Kasus: Pencarian Kunci Menuju Pintu Gerbang

Pengantar Kecerdasan Buatan — Knowledge-Based Agent

Nama Kelompok: (isi nama kelompok)

Anggota & NIM:

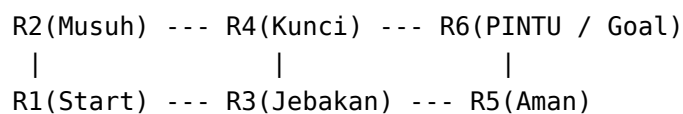
1. (Nama) — (NIM)
2. (Nama) — (NIM)
3. (Nama) — (NIM)

1. Deskripsi Masalah

Studi kasus ini merupakan varian dari Wumpus World: seorang Agent ditempatkan pada peta ruangan (dungeon) dan harus mencapai tujuan berupa membuka Pintu Gerbang. Namun, pintu tersebut terkunci — Agent harus terlebih dahulu menemukan Kunci yang berada di salah satu ruangan, sebelum bisa membuka pintu. Selain itu, terdapat Musuh dan Jebakan di beberapa ruangan sebagai elemen tambahan yang memancarkan indikasi tidak langsung (persepsi) ke ruangan tetangganya.

Berbeda dari Wumpus World asli — yang membutuhkan penyimpulan bentuk negatif ("ruangan ini aman") dan karenanya tidak bisa diproses forward/backward chaining murni — seluruh aturan pada studi kasus ini sengaja dirancang sebagai Horn Clause (kesimpulan tunggal, positif, tanpa negasi), sehingga valid diproses oleh kedua metode inferensi tersebut.

Peta Ruang



2. Representasi Pengetahuan (Knowledge Base)

2.1 Predikat yang Digunakan

Predikat	Arity	Arti
Terhubung(x, y)	Binary	Ruangan x terhubung ke ruangan y
DiRuangan(Agent, x)	Binary	Agen sedang berada di ruangan x
AdaKunci(x)	Unary	Ruangan x berisi kunci
AdaMusuh(x)	Unary	Ruangan x berisi musuh
AdaJebakan(x)	Unary	Ruangan x berisi jebakan
Berkilau(x)	Unary	Persepsi: ada kilauan di x (indikasi kunci di tetangga)
Berbau(x)	Unary	Persepsi: ada bau busuk di x (indikasi musuh di tetangga)
Berangin(x)	Unary	Persepsi: ada angin di x (indikasi jebakan di tetangga)
MemilikiKunci(Agent)	Unary	Agen sudah membawa kunci
PintuTerbuka()	Nullary	Goal akhir — pintu berhasil dibuka

2.2 Fakta Awal (Initial KB)

Terhubung (R1, R2)	Terhubung (R1, R3)	Terhubung (R2, R4)	
Terhubung (R3, R4)	Terhubung (R3, R5)	Terhubung (R4, R6)	Terhubung (R5, R6)
AdaKunci (R4)	AdaMusuh (R2)	AdaJebakan (R3)	
DiRuangan (Agent, R1)			

2.3 Aturan (Rules) — Seluruhnya Horn Clause

Kode	Aturan
R1	Terhubung(x,y) -> Terhubung(y,x)
R2	AdaKunci(x) ^ Terhubung(x,y) -> Berkilau(y)
R3	AdaMusuh(x) ^ Terhubung(x,y) -> Berbau(y)
R4	AdaJebakan(x) ^ Terhubung(x,y) -> Berangin(y)
R5	DiRuangan(Agent,x) ^ AdaKunci(x) -> MemilikiKunci(Agent)
R6	MemilikiKunci(Agent) ^ DiRuangan(Agent,R6) -> PintuTerbuka()

3. Formulasi Goal

Goal yang ingin dibuktikan pada studi kasus ini adalah PintuTerbuka() — yaitu suatu kondisi yang hanya bisa dicapai jika Agent (a) sudah membawa kunci (MemilikiKunci(Agent)), dan (b) sedang berada di ruangan R6

(DiRuangan(Agent, R6)), sesuai aturan R6. Kedua metode inferensi — forward chaining dan backward chaining — akan diuji apakah dapat membuktikan goal ini pada dua kondisi KB yang berbeda (lihat Bab 5: Eksperimen).

4. Implementasi Forward Chaining

Forward chaining diimplementasikan secara generik menggunakan proses unifikasi (bukan hanya pencocokan proposisi sederhana), sehingga dapat menangani aturan bervariabel seperti Terhubung(?x,?y) -> Terhubung(?y,?x). Algoritma bekerja secara bertahap (iteratif): pada setiap iterasi, seluruh aturan dicoba dicocokkan terhadap fakta yang ada di KB; setiap kecocokan menghasilkan fakta baru yang ditambahkan ke KB. Proses ini diulang sampai tidak ada fakta baru yang muncul (fixpoint).

```
def forward_chaining(facts, rules, max_iterations=20):
    kb = set(facts)
    ... # loop sampai fixpoint, cocokkan premis via unifikasi
    return {facts, trace, rules_fired, iterations}
```

Output program mencatat setiap fakta baru yang berhasil diturunkan beserta aturan dan substitusi yang menghasilkannya (trace), serta metrik rules_fired (jumlah total aturan yang berhasil tertembak) dan iterations (jumlah putaran sampai fixpoint).

5. Implementasi Backward Chaining

Backward chaining diimplementasikan sebagai fungsi rekursif goal-driven: dimulai dari goal yang ingin dibuktikan, algoritma mencoba mencocokkan (unifikasi) goal tersebut langsung dengan fakta yang ada di KB; jika tidak ditemukan, algoritma mencari aturan yang kesimpulannya cocok dengan goal, lalu mencoba membuktikan seluruh premis aturan tersebut secara rekursif.

Setiap kali suatu aturan dipakai dalam pencarian rekursif, variabelnya di-standardize apart (diberi nama unik) untuk mencegah tabrakan substitusi antar pemanggilan rekursif yang berbeda. Algoritma ini diimplementasikan sebagai generator Python, sehingga dapat menelusuri lebih dari satu jalur pembuktian jika ada.

```
def backward_chaining(facts, rules, goal, subst, depth, trace):
    # 1) coba cocokkan goal langsung dengan fakta
    # 2) coba cocokkan goal dengan kesimpulan tiap aturan,
    #    lalu buktikan seluruh premisnya secara rekursif
```

Metrik yang dicatat adalah goals_checked (jumlah goal/subgoal yang diperiksa sepanjang pencarian) dan proven (berhasil/tidaknya goal dibuktikan).

6. Eksperimen

Skenario 1 — Kondisi Awal (Agent di R1, belum bergerak)

Pada kondisi ini, Agent masih berada di titik Start (R1) dan belum pernah berpindah ruangan.

Metode	Hasil	Metrik
Forward Chaining	GAGAL membuktikan PintuTerbuka()	15 rules fired
Backward Chaining	GAGAL membuktikan PintuTerbuka()	4 goal dicek

Forward chaining tetap menembak 15 aturan — termasuk menurunkan seluruh fakta Berkilau, Berbau, dan Berangin di berbagai ruangan — meski tidak satupun relevan langsung dengan goal. Backward chaining jauh lebih hemat: hanya 4 goal yang diperiksa, karena langsung fokus menelusuri rantai PintuTerbuka() <- MemilikiKunci(Agent) <- DiRuangan(Agent,x) ^ AdaKunci(x), dan segera berhenti begitu diketahui AdaKunci(R1) tidak pernah benar.

Skenario 2 — Setelah Agent Bergerak (TELL posisi baru)

Pada skenario ini, KB diperbarui secara bertahap mensimulasikan TELL saat Agent berpindah: pertama ke R4 (mengambil kunci), lalu ke R6 (mencapai pintu). Rute perpindahan (R1 -> R2 -> R4, lalu R4 -> R6) diperoleh melalui pencarian BFS sederhana pada graph Terhubung.

Metode	Hasil	Metrik
Forward Chaining	BERHASIL membuktikan PintuTerbuka()	17 rules fired total (16 saat sampai R4 + 1 saat sampai R6)
Backward Chaining	BERHASIL membuktikan PintuTerbuka()	5 goal dicek

Menariknya, saat proses pengembangan implementasi ini, ditemukan sebuah kasus frame problem: fakta MemilikiKunci(Agent) sempat "hilang" ketika posisi Agent diperbarui dari R4 ke R6, karena pembaruan posisi secara naif hanya mengganti fakta DiRuangan tanpa mempertahankan fakta lain yang sudah benar sebelumnya. Solusinya: KB hasil forward chaining pada tahap sebelumnya (yang sudah memuat MemilikiKunci(Agent)) dipakai sebagai basis pembaruan posisi berikutnya, sehingga fakta yang relevan tetap terbawa.

7. Perbandingan Forward vs Backward Chaining

Aspek	Forward Chaining	Backward Chaining
Arah penalaran	Fakta -> kesimpulan	Goal -> fakta
Sifat	Data-driven	Goal-driven
Skenario 1 (gagal)	15 rules fired	4 goal dicek
Skenario 2 (berhasil)	17 rules fired total	5 goal dicek
Cocok untuk	Menyimpulkan semua hal dari fakta yang ada	Menjawab satu pertanyaan spesifik

8. Analisis dan Refleksi

Bagian ini dapat diisi dengan jawaban atas pertanyaan analisis, misalnya:

1. Mengapa forward chaining tetap menembak 15 aturan di Skenario 1 meski goal akhirnya gagal dibuktikan?
2. Mengapa backward chaining jauh lebih hemat (goal dicek lebih sedikit) dibanding forward chaining pada kedua skenario?
3. Apa yang dimaksud dengan frame problem, dan bagaimana hal itu muncul pada implementasi studi kasus ini?
4. Mengapa Wumpus World versi asli tidak bisa diproses forward/backward chaining murni, sementara studi kasus modifikasi ini bisa?
5. Berdasarkan hasil eksperimen, metode mana yang lebih sesuai untuk sistem yang perlu menjawab pertanyaan spesifik secara berulang (misalnya sistem tanya-jawab), dan metode mana yang lebih sesuai untuk sistem yang reaktif terhadap event/fakta baru secara terus-menerus?

9. Kesimpulan

Ringkas temuan utama: forward chaining dan backward chaining sama-sama valid secara logika (keduanya berhasil membuktikan goal pada Skenario 2), namun berbeda signifikan dalam efisiensi tergantung kebutuhan — backward chaining lebih unggul untuk menjawab pertanyaan spesifik, sedangkan forward chaining lebih cocok untuk sistem yang perlu menyimpulkan seluruh implikasi dari fakta yang terus bertambah. Studi kasus ini juga menyingkap keterbatasan penting inferensi logika murni: perubahan state (seperti posisi Agent) tidak bisa disimpulkan begitu saja dari KB statis, melainkan harus direpresentasikan lewat mekanisme TELL yang eksplisit.